

Report Till 1–5 Weeks

Project MAESTRO — Design and Evaluation of a Safe Multi-Agent System for Natural Language-Driven Desktop Task Automation. Group 298, B.Tech 7CSE (Evening), Amity School of Engineering and Technology, session 2023–27. Submitted by Shashank Gupta (A2345923073), Seenu (A2345923074) and Jairaj Berry (A2345923013), under the guidance of Dr. Rajni Sehgal Kaushik. Project area: Agentic AI with specialization in Natural Language Processing.

Introduction

This report summarises the work carried out in the first five weeks of the minor project, as recorded in Weekly Progress Reports 1 to 5. The project aims to build a desktop assistant that accepts an ordinary English instruction, such as moving a set of files or organising a folder, and carries it out on a real computer. The difference from the systems that already exist is that our system does not execute an instruction blindly. Every action it plans is first written out in a structured form, checked against a set of safety rules, simulated without touching the disk, and shown to the user for approval before anything actually happens. The first five weeks were spent establishing what the problem is, what has already been done by others, where the gap lies, and what our system is therefore required to do.

Week 1 — Project Definition and Feasibility

In the first week the project title was finalised as "Design and Evaluation of a Safe Multi-Agent System for Natural Language-Driven Desktop Task Automation", and the synopsis was submitted. The objectives and the scope of the work were fixed, and the research domains the project draws from were identified, namely Artificial Intelligence, Natural Language Processing, Multi-Agent Systems, Desktop Automation and AI Safety. A technical and economic feasibility study was carried out. The project was found technically feasible because it can be built entirely with existing open-source AI models, freely available frameworks and ordinary desktop hardware, and economically feasible because the entire software stack is free and open source, giving a bill of material of zero. The eight-phase project schedule was also drawn up in the form of a PERT chart, covering literature review, study of existing systems, requirement analysis, architecture design, module development, integration, testing and final documentation.

Week 2 — Literature Review (Part I)

The second week was spent on the literature survey. We studied the current state of research on AI agents, Natural Language Processing and Large Language Models, in order to understand how such systems are presently built and evaluated. Research papers were collected and organised, the basic concepts behind AI agents were reviewed, and the architecture of Large Language Models was studied in some detail so that we could understand how an instruction

given in plain English is converted into a sequence of machine actions. The outcome of this week was the first part of the Literature Review chapter of the report.

Week 3 — Study of Existing Systems and Comparative Study

In the third week the survey moved from general theory to the actual systems that already perform desktop automation. We studied OpenClaw, Hermes and Open Interpreter, and compared their architectures and capabilities against one another. The comparison showed that these systems are already strong at understanding open-ended natural language and at carrying out tasks on a real machine, so raw capability is not the weakness. What they lack is a way of representing a planned action in a form that can be inspected before it runs, any notion of which actions are dangerous and which are harmless, any preview of the effect of an action, and any reliable way of undoing what has been done. This week produced the Comparative Study section of the report.

Week 4 — Research Gap and Novelty Statement

The fourth week was devoted to the research gap analysis. Working from the comparison prepared in the previous week, we identified the specific shortcomings of the existing systems, which fall into three areas: transparency, safety and modular task execution. Existing work reports how capable an agent is but almost never reports how safe it is, so there is no accepted way of comparing two desktop agents on the risk they carry. On this basis the contribution of the project was defined and the novelty statement was finalised. Our contribution is that every planned action is compiled into a typed and inspectable intermediate representation, is scored for risk by fixed program logic that the language model cannot influence, is simulated before it runs, is gated behind a confirmation proportionate to its consequence, and is written to a log that cannot be altered without detection. This week produced the Research Gap chapter of the report.

Week 5 — Requirement Analysis

In the fifth week the work moved from research to specification. Requirement gathering was carried out, the situations in which a user would actually use the system were analysed, and the functional requirements were identified and written down. The user scenarios covered the categories of work the system is expected to handle, which are file and folder management, search and retrieval, browser tasks, application control, system information, and drafting of documents and messages. The actions the system must refuse outright were also listed at this stage, such as making payments, entering passwords, deleting anything permanently without a recovery path, and changing the security settings of the operating system. The outcome of this week was the Requirement Analysis of the project.

Work Done on the Implementation Side

Alongside the weekly research work, we have also begun building the system, so that the eighth semester does not start from an empty repository. A working version of the software already exists and runs offline on the project machine. It accepts an English instruction, converts it into a structured plan using a language model running locally, checks that plan against the safety rules, scores the risk of each step, simulates the effect of the plan without changing anything on the disk, shows the result to the user for approval, and only then executes it while recording every step in a log. Seven file-handling operations have been implemented, each with its own simulation and undo behaviour. Forty-eight automated tests are currently passing, including tests that deliberately attempt to defeat the safety layer: a plan that tries to escape the permitted folders is refused, a plan that falsely declares itself harmless is still scored correctly by the program logic, and any attempt to edit the log afterwards is detected. The entire system runs on free and open-source software at no cost.

Present Status

At the end of the fifth week the project stands as follows. The literature review, the comparative study of existing systems and the research gap analysis are complete, and the requirement analysis has been prepared. The corresponding chapters of the report, namely the Literature Review, the Comparative Study and the Research Gap, have been drafted, and five Weekly Progress Reports have been filed. On the implementation side the core of the system is already working and tested, which is ahead of the schedule laid out in the PERT chart. The work planned for the coming weeks is to freeze the specification of the action representation and the risk classification, to prepare the system architecture along with the required design diagrams, to fix the evaluation methodology and the metrics by which the system will be judged, and to begin building the dataset of instruction and plan pairs that the system will later be trained on.